

Tutorial 6 – Files and exception handling (Submit all code for Question 1)

Question 1 (Adapted from previous year final exam)

Figure Q-1 below illustrates the content of an “AmpRes.txt” text file, which contains a list of ampere (I) and corresponding resistance (R) readings from a set of electrical devices.

2.25	200.5
1.54	325.5
2.84	180.6
2.10	210.1
4.36	90.7
4.89	81.5
2.55	190.85

Figure Q1: AmpRes.txt file.

Using the C++ programming language, create a class, **CPowerInfo**, which loads a set the ampere and resistance readings from the aforementioned file into memory and calculates the dissipated power of each element. In this class, create the following:

- A class member structure called struct **SPowerData**, which stores a set of ampere, resistance and power floating point values.
- A member function **loadData(string pFile, int numElements)** that loads the content of “AmpRes.txt” file into a class member array of type **struct SPowerData**. This function also computes the dissipated power (*in watts*) of each array element and saves computed power into this array.

This function takes in two input arguments. The first argument represents a constant string of the text file name to load and the second argument represents the number of elements in the text file. This function returns void. The C++ **fstream** class must be used here in reading from the file.

- A member function **void FindMinMaxPower(float &outMinPower, float &outMaxPower)** that determines the minimum and maximum power values in the class member array of type **SPowerData**.

Note: Write a complete C++ class based on the requirements above and declare the necessary class member variables. Remember to include the necessary header files in your code. The driver program **main()** to test this class is given.

Formula for power calculation (in watts) = (Ampere(I))² × Resistance (R)

```

int main() {
CPowerInfo pw;
int numElements = pw.getNumElements("AmpRes.txt"); // try implement this function
pw.LoadData("AmpRes.txt", numElements);
float minpower, maxpower;
pw.FindMinMaxPower(minpower, maxpower);
cout << "\n The minimum power = " << minpower;
cout << "\n The maximum power = " << maxpower;
cout << endl;

    return 1;
}

```

```

The minimum power = 771.956
The maximum power = 1948.84

```

Starter code

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

```

```

class SPowerData
{
public:
    float m_Amp;
    float m_Res;
    float m_Power;
};

```

```

class CPowerInfo
{
private:
    //
    SPowerData *m_power_array; // why pointer ? to store array of struct variables
    fstream m_File;
    int numElem;

```

```

public:

```

```

// member functions based on question
void LoadData(string pFile, int numElements)
{
    m_File.open(pFile);

    cout << "\n Total elements :" << numElements;
    float d1, d2;
    for (int i = 0; i < numElements; i++)
    {
        m_File >> d1;
        m_File >> d2;
        cout << "\n"
             << d1 << "\t" << d2;
    }
    m_File.close();
}
//
};

int main()
{
    cout << "Test program!" << endl;
    CPowerInfo c1;
    c1.LoadData("AmpRes.txt", 7);
    return 1;
}

```

Question 2

Improve the program in question to handle wrong file name selection with the use of exception handling. The throw command to be executed if the file is not found. The try-catch block is in the main() function.

Question 3 (Final exam 2020/21)

Figure Q1-1 illustrates a Unified Modeling Language (UML) class diagram for two classes, namely **Employee** and **Company**. All member functions must be implemented.

The class **Company** contains member functions that reads data about the list of staff and their personal salary from two text files, namely staff.txt and salary.txt, respectively, as shown in Figure Q1-2. The data will then be displayed on screen. The two text files represent data extracted from a sql database. The **Employee** class is used to store information about the individual staff in the company. Detail explanation of the classes member functions is available in Table Q1-1 and Table Q1-2

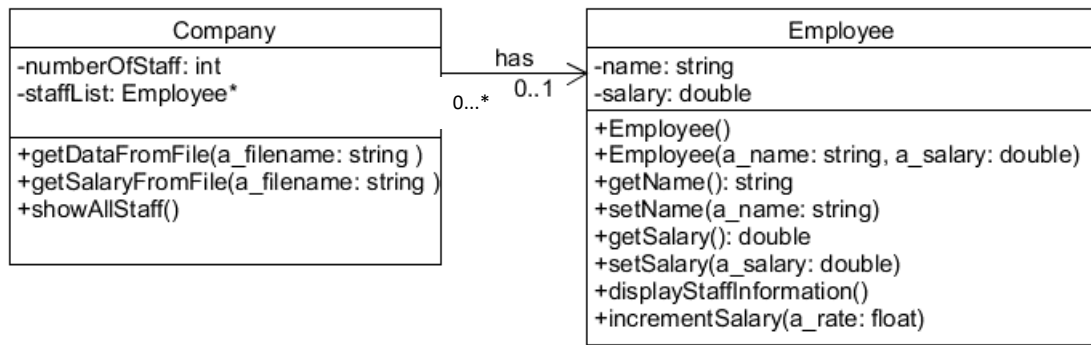
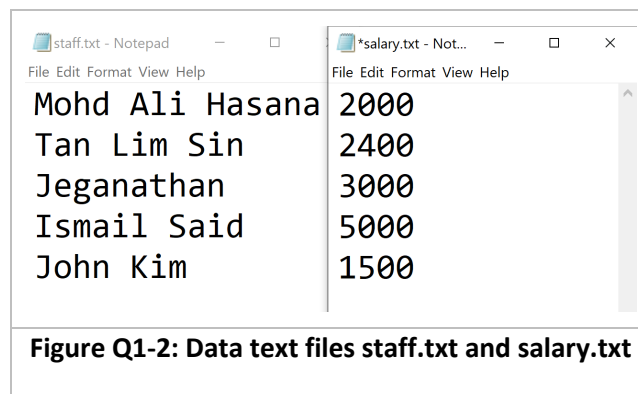


Figure Q1-1: Class UML Diagram.



The driver function that tests the member functions in both the classes is listed in Figure Q1-3. The corresponding output is shown in Figure Q1-4

```

int main() {
    Employee e1 = Employee("Nurazlan", 1800);
    e1.displayStaffInformation();
    e1.incrementSalary(0.20);
    cout << "\nAfter salary increment";
    e1.displayStaffInformation();
    Employee e2 = Employee("Donald James", 3500);
    e2.incrementSalary(0.20);

    Company astro;
    cout << "\n\nReading staff name from text file ";
    astro.getDataFromFile("staff.txt");
    astro.showAllStaff();
    cout << endl;
    cout << "\nReading salary from text file ";
    astro.getSalaryFromFile("salary.txt");

    cout << "\n\n" ;
}

```

Figure Q1-3

```

Name : Nurazlan, Salary = RM 1800
After salary increment
Name : Nurazlan, Salary = RM 2160
Donald James not qualified for salary increment

Reading staff name from text file
List of staff
Mohd Ali Hasana
Tan Lim Sin
Jeganathan
Ismail Said
John Kim
Number of staff = 5

Reading salary from text file
Mohd Ali Hasana salary in RM :2000
Tan Lim Sin salary in RM :2400
Jeganathan salary in RM :3000
Ismail Said salary in RM :5000
John Kim salary in RM :1500

```

Figure Q1-4

- a) **Define** the two classes **Company** and **Employee** based on the UML class diagram shown in Figure Q1-1. [6 marks]

- b) Implement all the member functions for the class **Employee**. The member functions must provides the correct outputs based on the **main()** driver function shown in Figure Q1-3 and the output is shown in Figure Q1-4. The explanation of the corresponding functions is listed in Table Q1-1 below.

Table Q1-1

No	Function name	Brief Explanation
1	<code>Employee(string a_name, double a_salary)</code>	Constructor to initialize the staff name and their salary based on <code>a_name</code> and <code>a_salary</code> . (2 marks)
2	<code>string getName()</code>	Return the name of the staff (1 mark)
3	<code>setName(string a_name)</code>	Set the name of the staff based on <code>a_name</code> (2 marks)
4	<code>getSalary()</code>	Return the salary of the staff (1 mark)
5	<code>setSalary(double a_salary)</code>	Set the salary of the staff based on <code>a_salary</code> (2 marks)
6	<code>displayStaffInformation()</code>	Display staff name and salary (2 marks)
7	<code>incrementSalary(float rate)</code>	Increase the staff salary based on <code>rate</code> (in %) if the salary is below RM2000 . It displays the relevant message if the staff is not qualified for increment. (2 marks)

[12 marks]

- c) Implement all the member functions for the class **Company**. The member functions must provides the correct outputs based on the **main()** driver function shown in Figure Q1-3 and the output shown in Figure Q1-4. The explanation of the corresponding functions is listed in Table Q1-2 below.

Table Q1-2

No	Function name	Brief Explanation
1	<code>getDataFromFile(string filename)</code>	Read the staff names based on input filename and store the names in an array of Employee objects. The <code>numberOfStaff</code> value is updated based on the total number of staff names read from the text file filename. Use the <code>getline()</code> function to read the text file line by line. (2 marks)
2	<code>getSalaryFromFile(string filename)</code>	Read the staff salaries based on input filename and store the salaries in the array of Employee objects salary attribute. All the staff name and their salary is then displayed as shown in Figure Q1-4. You can use the <code>stof()</code> function to convert string to floating point numbers. (2 marks)
3	<code>showAllStaff()</code>	Display all the staff names. The number of staff will then be displayed. (2 marks)

[6 marks]

